
ENRICHLY HR, INC. • ENGINEERING

Full Stack Developer Intern Engineering Assignment

Build: A Job Automation Platform

Version 1.0 | August 2026
Enrichly HR, Inc.



What we want you to build

Build a web application where users can create and manage automated jobs.

A job could be something like:

- Call an API every hour
- Sync data between two systems
- Trigger a webhook
- Run some background process on a schedule

Users should be able to create jobs, run them, see their execution history, and understand what happened when something fails. That's the basic idea.

An open-ended problem, on purpose

We are intentionally leaving a lot of the product undefined. We don't want to give you a list of screens and APIs and ask you to implement them exactly as written.

We want to see how you approach an open-ended engineering problem.

You should decide

- What the UI should look like
- What information a user actually needs
- How jobs and executions should be represented
- How retries should work
- What happens when something fails
- What the dashboard should show
- How you structure the database and APIs
- How the system handles multiple workers
- What happens when parts of the system go down

If you think there is something important missing from this assignment, add it. If you think something here doesn't make sense, change it. We are interested in the decisions you make and the reasoning behind them.

Technical Requirements

Our preferred stack is:

Next.js + React + TypeScript + .NET/C# + PostgreSQL

You can use a different stack if you have a good reason. We are more interested in how you build the system than in following a specific technology list.

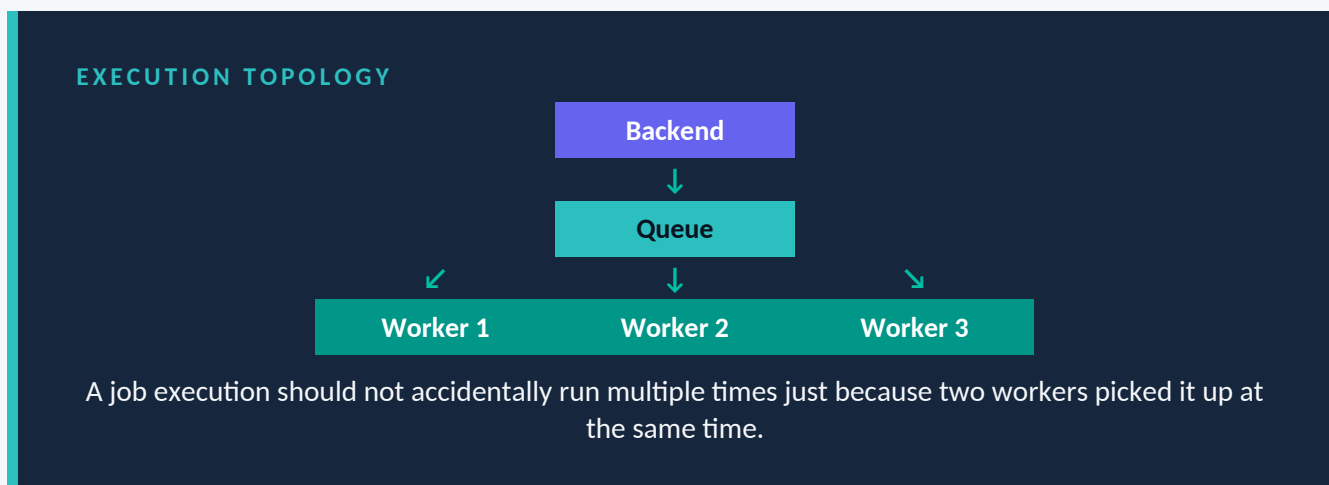
The application should have

- A frontend
- A backend API
- A relational database
- A background worker / job execution system
- A publicly accessible deployment

Docker is recommended.

The Important Part: Job Execution

Your system should support multiple workers processing jobs. For example:



Think about what happens when

- A worker crashes while processing a job
- An API request times out
- An external API is unavailable
- The network connection drops
- The database is temporarily unavailable
- A user clicks "Run Now" multiple times
- Two requests try to update the same job at the same time
- A job fails halfway through its execution

You don't need to build a distributed-systems masterpiece. We do expect you to identify the important failure cases and make reasonable decisions about them.

Product

Please don't treat this as a simple CRUD application. Think about what a developer would actually want from a job automation tool.

At a minimum, the product should make it possible to

- Create and manage jobs
- Run jobs
- See execution history
- Understand whether an execution succeeded or failed
- Retry failed executions

Beyond that, it's up to you

You might find things like these useful:

- Search and filtering
- Logs
- Cancellation
- Notifications
- System / worker health
- Real-time execution updates
- Better failure details
- Job statistics

You don't need to build all of them.

A small product with good decisions is better than a large product with lots of half-finished features.

Backend

The backend should be structured like something you would be comfortable maintaining.

It should include the usual pieces you'd expect in a real application

- Authentication
- Authorization

- Request validation
- Error handling
- Database migrations
- Logging
- Background processing
- API documentation

Pay particular attention to

- Concurrency
- Duplicate executions
- Retries
- Job / execution state transitions
- Worker failures
- Stale jobs
- Idempotency
- Database and queue failures

You don't have to solve every problem perfectly. Just make your assumptions clear and handle the important cases.

Testing

Write automated tests for the parts that matter. We aren't looking for 100% code coverage.

Focus on areas where bugs would be painful, especially

- Concurrent job execution
- Retries
- Authorization
- Failure recovery
- Duplicate requests
- State transitions

A few good tests are more useful than a large number of superficial tests.

Deployment

Deploy the application somewhere we can actually use it.

We should be able to

1. Open the frontend
2. Create or log in as a user
3. Create a job
4. Trigger a job
5. See its execution
6. See failures and retries
7. Use the application without setting it up locally

Please provide

- Live URL
- GitHub repository
- README
- Environment variable documentation
- Local setup instructions
- Deployment instructions

The deployed application should be working when you submit it.

Engineering Notes

Add an ENGINEERING.md file to the repository. Keep it reasonably short — we want to understand your thinking, not read a 20-page architecture document.

Include

- A quick overview of the architecture
- How jobs are picked up and executed
- How you handle concurrency
- How retries and failures work
- Important database decisions
- Product decisions you made
- Known limitations
- What you would improve if you had more time

If you made a trade-off because of the time limit, mention it.

AI Usage

You are free to use AI tools. Use Claude Code, Codex, Cursor, GitHub Copilot, ChatGPT, Gemini, or whatever tools you normally use.

We are not trying to test whether you can type code without assistance. What we do care about is whether you understand what you built. You should be able to explain your architecture, your important decisions, and the trade-offs you made.

If the submission is largely AI-generated and you can't explain how the system works or why you made certain decisions, that will be a problem.

Use AI as much as you want. Just make sure you own the final result.

What We'll Look At

There isn't a strict scoring formula, but these are the things we'll pay attention to:

CRITERION	WHAT WE'RE LOOKING FOR
Problem solving	Did you identify problems that weren't explicitly mentioned?
Product thinking	Did you build something that actually feels useful?
Architecture	Does the system make sense when multiple workers and failures are involved?
Code quality	Is the code easy to understand and maintain?
Database & API design	Do the models, APIs, indexes, and state transitions make sense?
Frontend	Is the application intuitive and easy to use?
Testing	Did you test the parts that are most likely to break?
Deployment	Does the application actually work when we open it?

There Is No "Correct" Solution

Two people could approach this assignment very differently and both could do a great job. Don't try to guess what implementation we are expecting.

Think through the problem, make reasonable decisions, and build something you would actually be comfortable shipping.

Time

Please aim to spend around

8-12 hours

on the assignment. Don't spend the entire time making the UI perfect.

A ROUGH PRIORITY WOULD BE

**Problem solving → Architecture → Correctness → Reliability →
Testing → UX**

You don't need to finish every possible feature. If you run out of time, ship the smaller version properly and document what you would do next.

We care much more about how you approached the problem and the quality of your decisions than how many features you managed to squeeze in.